Self Balancing Robot

Objective:

The primary objective is to design and construct a two-wheeled robotic vehicle that can maintain an upright position without external support.

Specific goals include:

- Mechanical Design: To build a chassis where the Center of Mass (COM) is balanced above the wheel axis.
- Sensor Fusion: To combine data from an accelerometer and gyroscope (using an MPU6050) to obtain a noise-free, accurate tilt angle.
- Closed-Loop Control: To implement a PID (Proportional-Integral-Derivative) algorithm that continuously calculates and applies the precise motor speed required to counteract gravity."

Project Motivation (Why?) :

The motivation behind this project was to bridge the gap between static programming and real-world dynamic systems. While standard robotics projects (like obstacle avoiders) operate in stable environments, a self-balancing robot represents an inherently unstable system.

I wanted to challenge myself to understand Control Theory—specifically how to maintain stability in a system that constantly wants to fail (fall over). This project serves as a practical application of the 'Inverted Pendulum' problem, a classic benchmark in control engineering used to test algorithms that are eventually applied in real-world technologies like Segways, rockets (gimbaled thrust), and bipedal humanoid robots.

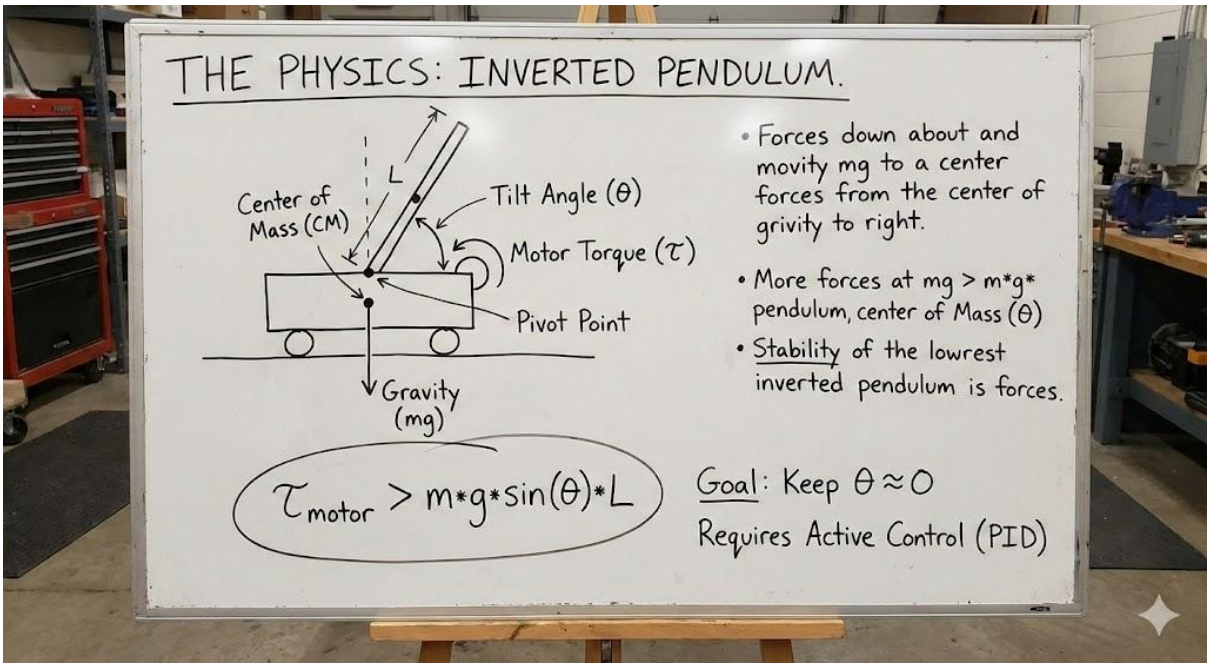
Physics behind the project:

The Physics of the Inverted Pendulum

The Analogy: The Broomstick on the Palm

Imagine trying to balance a broomstick upright on the palm of your hand.

- If the broom leans forward, you must instinctively run forward to "catch" it.
- By moving the base (your hand) under the center of mass (the top of the broom), you restore balance.
- The robot does exactly this, but using wheels instead of legs.



The Forces at Play

1. **Unstable Equilibrium:** The robot exists in a state of unstable equilibrium. It is perfectly balanced only when the Center of Mass (COM) is directly vertical above the Wheel Axis (Pivot Point).
2. **Gravitational Torque:** The moment the robot tilts by even 1 degree (let's say angle theta), gravity pulls down on the Center of

Mass. Because the COM is not directly over the wheels, this creates a **Torque** (a twisting force).

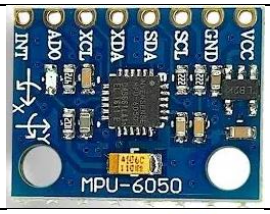
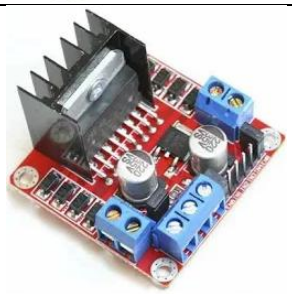
- *Formula:* tau = m \cdot g \cdot L \cdot \sin(\theta)$
- (Where m = mass, g = gravity, L = distance to center of mass, θ = tilt angle).
- *In plain English:* Gravity is trying to rotate the robot into the ground.

3. **Restoring Torque (τ_m):** To stop the fall, the motors must generate a counter-torque.
   - The motors drive the wheels *in the direction of the fall*.
   - This acceleration creates a reaction force that pushes the wheels "under" the falling center of mass.
   - For the robot to recover, the **Motor Torque** must be greater than the **Gravitational Torque**.

Components:

1. MPU6050
2. BO Motors - x2
3. Motor Driver (L293N)
4. 7.4v Battery

	Component Name	Image	Price
1.	Arduino Uno		₹200-₹300

2.	MPU6050		₹150-₹200
3.	BO Motors - x2		₹100
4.	Motor Driver (L293N)		₹200
5.	7.4v Battery		₹400

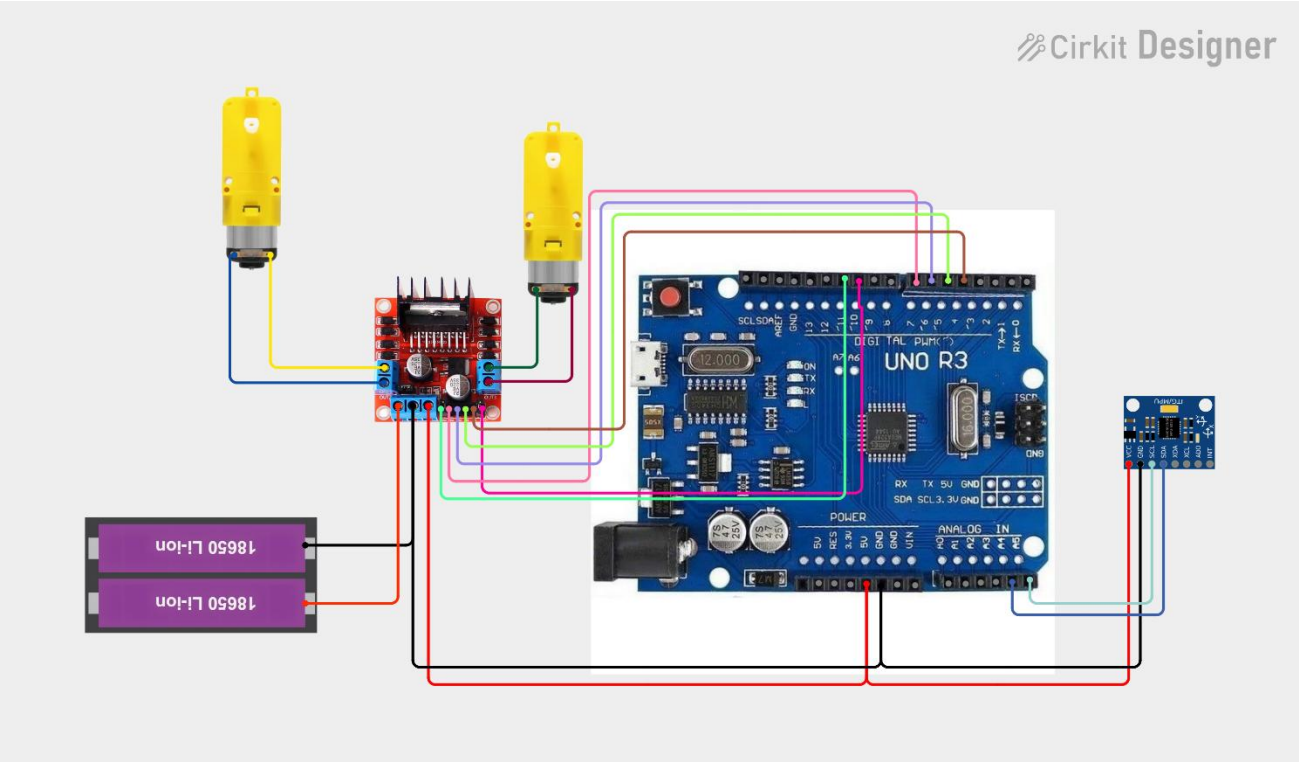
Code:

https://drive.google.com/file/d/1MrztL3tE2ZJ3OPQQZQ8GWYjd-vwuZwN8/view?usp=sharing

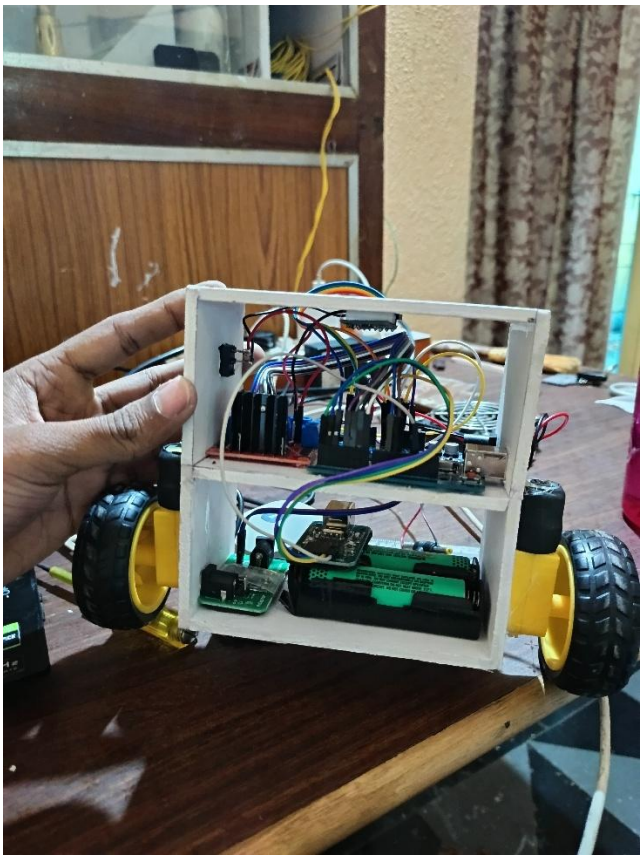
Version 2.0 (Bluetooth control):

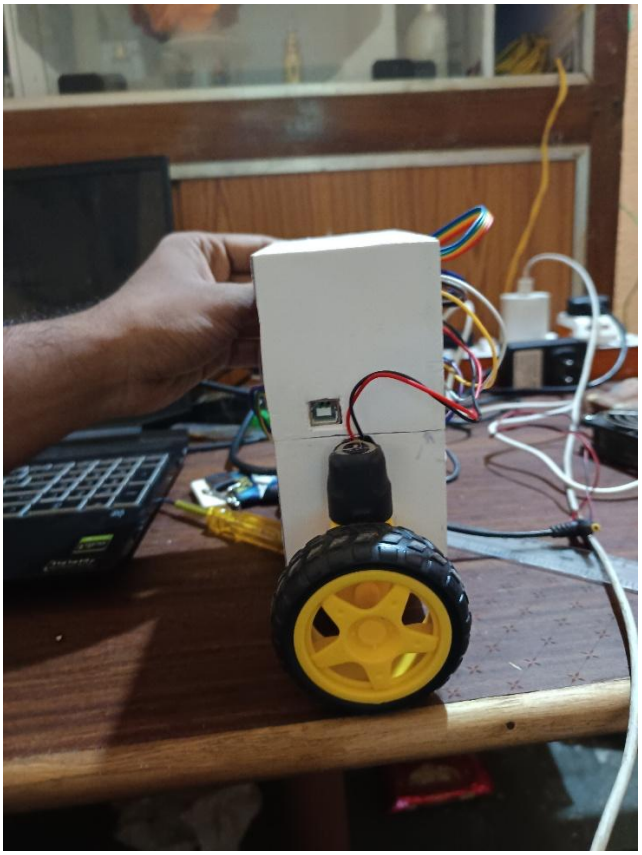
https://drive.google.com/file/d/1PlQH-FsJB8ggDSAgfRgIGTlxw6PcYEtx/view?usp=sharing

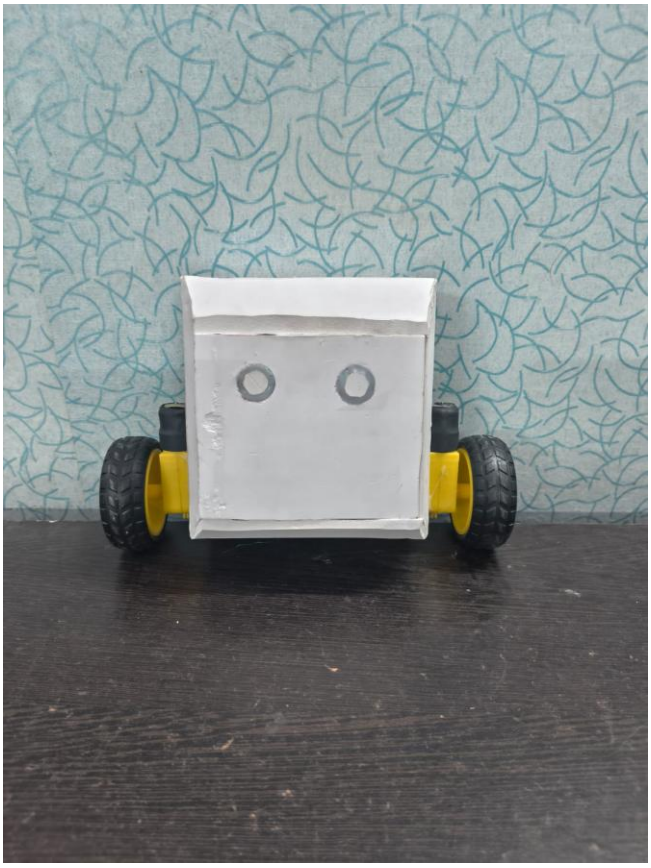
Circuit Connections:

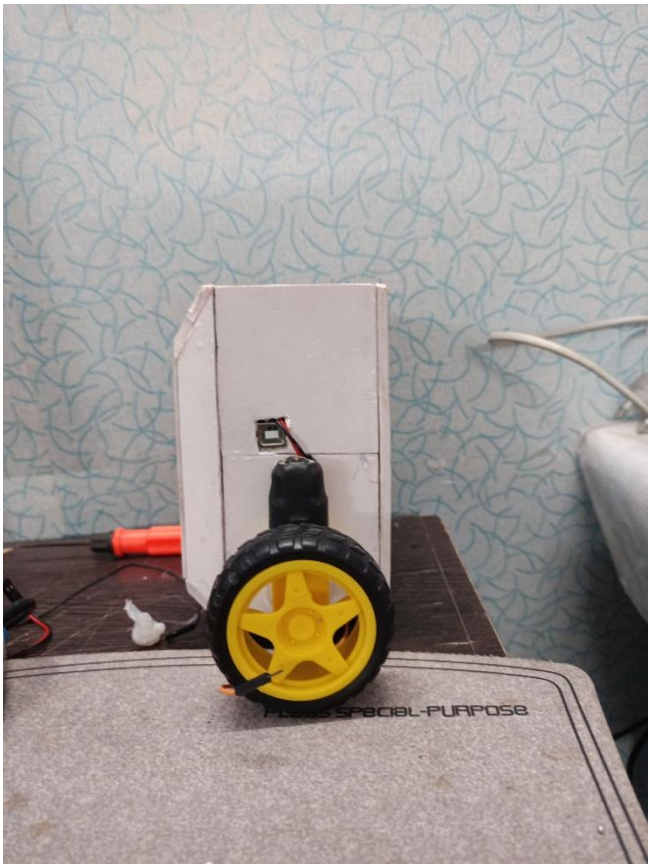


Photos:









Video:

Figure 3: Demonstration of the robot recovering from a push test and Carry Load. Click image to watch video (0:43).

Version 2.0: Added Bluetooth to control the robot.